DESIGN NOTE

AI Oversight: three-stage extraction pipeline

ai_oversight turns an AI-governance report (PDF or PPTX) into 22 canonical fields defined by PLAN.md — submission ID, model owner, risk rating, and so on — each with a value, a citation back to the source document, a status, and a confidence score. The pipeline runs in three stages, and the second and third each only do real work when the cheaper stage before them couldn't finish the job.

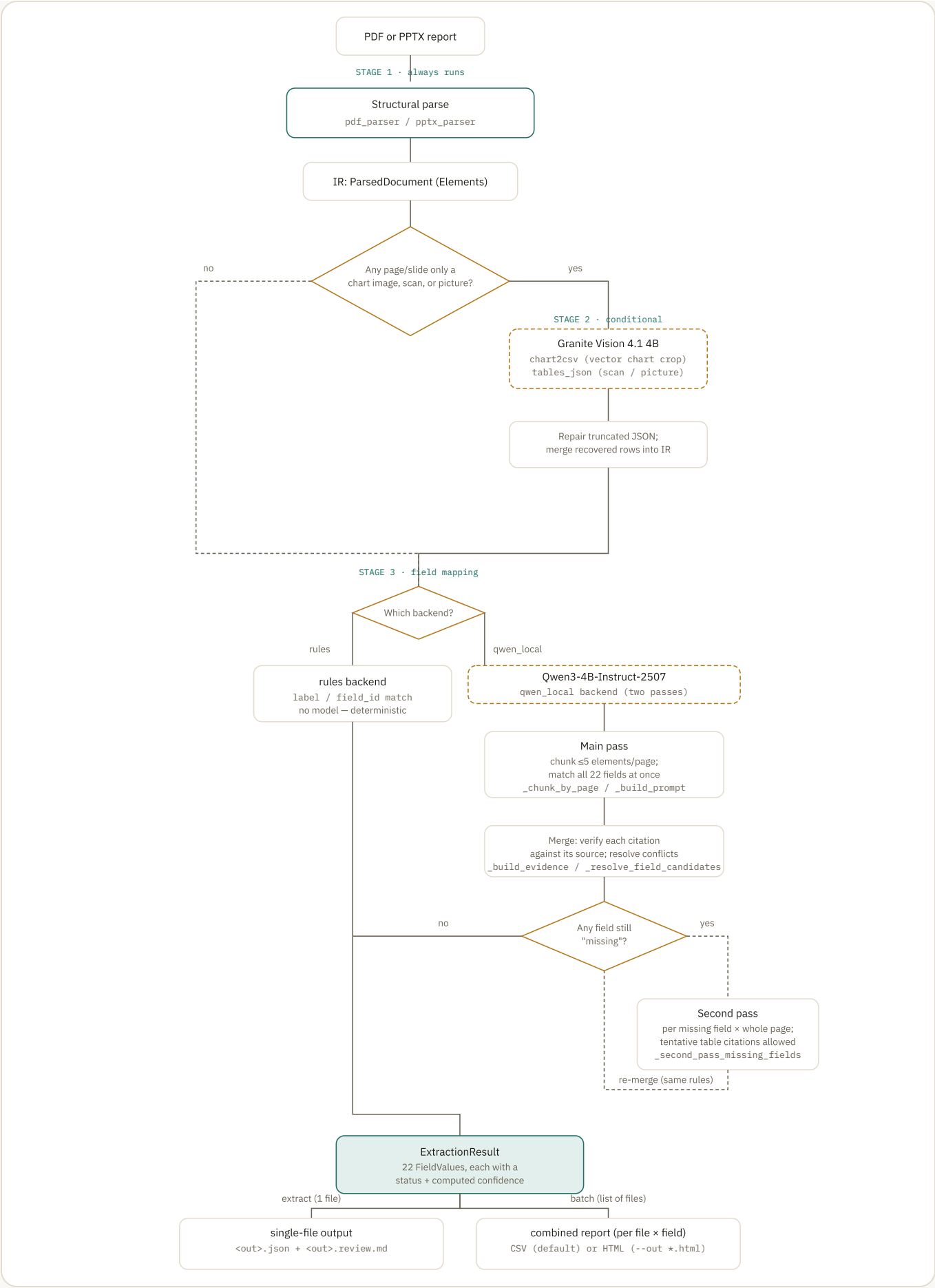
The organizing rule throughout is **never fabricate**: a value is only ever reported as confidently `extracted` when it can be traced to real text in the source document. Every other outcome — `needs_review`, `ambiguous`, `conflicting`, `missing` — exists so that uncertainty is visible to a human reviewer instead of silently guessed away.

Both models — Granite Vision 4.1 4B and Qwen3-4B-Instruct-2507 — are downloaded once from Hugging Face and run entirely on local hardware. There's no API call and no per-token cost to run `extract`, no matter how many times a document is processed or re-processed.

Hugging Face hosts many alternative models that could fill either role — larger, more capable ones included. Qwen3-4B-Instruct-2507 was chosen specifically for its small size: it keeps GPU and memory requirements low enough to run comfortably on ordinary local hardware (this project targets a 16GB Mac), rather than requiring a dedicated GPU server. Swapping in a different `AI_OVERSIGHT_TEXT_MODEL` is a config change, not a code change (see `Settings` below) — but every larger model trades away some of that low-resource portability.

The pipeline, end to end

Stage 1 is always cheap and deterministic. Stage 2 (Granite Vision) only runs on the specific pages/slides Stage 1 flagged as ambiguous — a vector-drawn chart, a scanned page, or a picture shape — never on content Stage 1 already parsed cleanly. Stage 3 maps whatever elements exist (native + vision-recovered) onto the 22 canonical fields, using either a fast deterministic backend or a two-pass local LLM backend.


PDF or PPTX report
STAGE 1 · always runs
Structural parse
pdf_parser / pptx_parser
IR: ParsedDocument (Elements)
Any page/slide only a chart image, scan, or picture?
no
yes
STAGE 2 · conditional
Granite Vision 4.1 4B
chart2csv (vector chart crop)
tables_json (scan / picture)
Repair truncated JSON; merge recovered rows into IR
STAGE 3 · field mapping
Which backend?
rules
qwen_local
rules backend
label / field_id match
no model — deterministic
Qwen3-4B-Instruct-2507
qwen_local backend (two passes)
Main pass
chunk ≤5 elements/page;
match all 22 fields at once
_chunk_by_page / _build_prompt
Merge: verify each citation against its source; resolve conflicts
_build_evidence / _resolve_field_candidates
Any field still "missing"?
no
yes
Second pass
per missing field × whole page;
tentative table citations allowed
_second_pass_missing_fields
re-merge (same rules)
ExtractionResult
22 FieldValues, each with a status + computed confidence
extract (1 file)
batch (list of files)
single-file output
<out>.json + <out>.review.md
combined report (per file × field)
CSV (default) or HTML (--out *.html)

Dashed outlines/edges mark the two conditional branches: Stage 2 only runs for flagged pages, and the second qwen_local pass only runs for fields the main pass left `missing`.

Stage 1 — Structural parse

`pdf_parser.parse()` (PyMuPDF) and `pptx_parser.parse()` (python-pptx) turn the raw file into the shared intermediate representation: a flat list of `Element`s, each one `text`, `key_value`, `table`, or `chart`. No model runs here — everything is regex, geometry, and shape structure.

Every stage exchanges data as Pydantic models — `Element`, `ParsedDocument`, `VisionObservation`, `ExtractionResult` — each one round-trips losslessly through `model_dump_json()` / `model_validate_json()`. That's deliberate: Stage 1's job is to capture everything the source document states, not to decide yet what's relevant to which of the 22 fields — that decision is Stage 3's alone. A saved `ParsedDocument` JSON carries the same information no matter which backend or model eventually reads it.

PDF specifics

Column-aware text blocks (the source reports are two-column); label→value pairing gated by on-page distance so a pairing is never guessed across unrelated text; a vector-drawn chart's gridline cluster is detected and its bounding region computed for Stage 2 to crop later.

PPTX specifics

Numbered-ID shape rows become `key_value` elements; non-ID multi-shape rows become `table` elements (ambiguous, multi-cell); a picture shape is never silently dropped — it's recorded as a warning that routes it to Stage 2.

This stage also decides, implicitly, whether Stage 2 has anything to do: a page that parsed cleanly carries no warning and is never sent to the vision model.

Stage 2 — Granite Vision (conditional)

`granite_vision.run_vision_pass()` only fires for exactly three situations Stage 1 could flag but not resolve on its own:

TRIGGER	TASK SENT TO GRANITE	WHY
A page has a vector-drawn chart (gridlines detected, not a native chart object)	`chart2csv`	PyMuPDF can see the strokes but not what they encode
A PDF page has no text layer and contains an image (fully scanned page)	`tables_json`	no native content exists to fall back on
A PPTX picture shape	`tables_json`	can't tell chart from table from a bare image, so the more general task is the default

Granite's raw response is repaired if the model's EOS token cut it off mid-table (`_repair_truncated_json` recovers every complete row before the cutoff), then merged back into the IR as ordinary `Element`s — a recovered chart becomes a `chart` element, a single clean instance row becomes a `key_value`, and a genuinely multi-instance table stays a cautioned `table` element. Whichever Stage 3 backend runs next never knows the difference between native and vision-recovered content.

This stage is **not** guaranteed lossless, in two distinct ways: a response cut off past `VISION_MAX_TOKENS` only yields the complete rows before the cutoff — the incomplete trailing row is dropped, not approximated; and a response that never parses at all (even after repair) contributes nothing to the IR, surfacing only as a warning. More fundamentally, for a scanned page or picture shape Granite's read of the image *is* the only source of truth — there's no native text layer to check it against. `ExtractionResult.vision_integrity` (a `computed_field`, derived purely from `vision_observations` so it can't drift out of sync) turns this into a document-level `pass` / `review` / `fail` rating: any outright-failed task drives it to `fail` regardless of how many other tasks on the same document were clean, a merely truncation-repaired task drives it to `review`, and it's `pass` only if every vision task recovered cleanly (or none ran at all). The `batch` report surfaces this per file, with the specific loss note for every non-clean task.

For a scanned PDF page specifically, a truncated `tables_json` response is not simply retried — generation here is deterministic (no temperature/sampling configured), and a real run confirmed retrying the identical whole-page call reproduces the identical truncated output. Instead, a lossy whole-page result triggers one retry: the page is split into top/bottom halves and each is queried separately, giving each call roughly half the content to output — the actual lever available to reduce how often the model stops early. This is a single-level retry, not recursive; a half that's still lossy after the split is accepted and reported as-is. Verified against the real model on the document that originally motivated this: a run that came back `review` (3/4 tasks clean, one truncated) now comes back `pass` (5/5 clean, the one lossy task replaced by its two clean halves), with the same 11/22 fields extracted either way.

Because this stage is slow (a real local vision model, also free of any API or token cost to run), its output can be cached once with the `vision` CLI command and replayed via `extract --`

`vision-cache` — Stage 3 can then be iterated on repeatedly without re-running Granite.

The same applies one level up: Stage 1's own output can be saved with `parse --out`, and that JSON is exactly the input every Stage 3 backend consumes. A saved parse — with or without a saved vision cache merged into it — can be re-matched by `rules`, by `qwen_local`, or by a future LLM backend, swapping which model does the matching without ever re-parsing the document or re-running Granite.

Stage 3 — Field mapping

Two interchangeable backends consume the same IR and produce the same `ExtractionResult` shape.

rules backend — deterministic

Matches each `key_value` / `table` element's own numbered `field_id` or label text against the canonical 22, with a synonym table for common wording variants. No model, no network, effectively instant — the right choice when the source document's own numbering is reliable.

qwen_local backend — two-pass local LLM

Runs **Qwen3-4B-Instruct-2507** locally via Hugging Face `transformers` — no API, no token cost — (greedy decoding — more on why below) in two passes:

① Main pass — chunked, multi-field

_chunk_by_page → _build_prompt → _extract_chunk → _build_evidence → _resolve_field_candidates

Each page is split into groups of at most 5 elements (`MAX_ELEMENTS_PER_CHUNK`) — asking a 4B model to match ~15 candidates against 22 fields in one call measurably hurt recall. Each chunk's prompt lists every canonical field's own PLAN.md description (not just its name) and warns that a `table`-type element describes one repeated instance, never the whole document. Every citation is checked against its source element's actual text before counting as substantiated, and a citation from a `table`-type element is *always* unsubstantiated — a structural rule, not a prompt request, because the model can choose to ignore a prompt instruction but can't bypass a code-level check.

② Second pass — per field, whole page

_second_pass_missing_fields → _build_single_field_prompt → _extract_single_field

For whatever fields are still `missing` after the main pass, ask about exactly *one* field at a time against the *entire* page's elements (no further sub-chunking — a single-field prompt has no multi-field output budget to protect). This pass's prompt also *invites* a tentative citation from a `table`-type element instead of forbidding it outright, since the structural check above already guarantees it can never outrank a verified answer — it can only ever surface as `needs_review`.

Both passes resolve candidates through the same merge rule: candidates are only ever compared for conflict *within* a confidence tier. A verified citation always wins over a disagreeing unverifiable one; a low-confidence table guess can never drag a correctly-substantiated answer down to `conflicting`.

Confidence score

Every field carries a `confidence` float — a triage signal for a reviewer, not a literal probability. It's a Pydantic `computed_field` on `FieldValue`, derived purely from `status` and `evidence`, so it can never drift out of sync with either one.

STATUS	BAND	MOVES TOWARD CEILING WHEN…
conflicting	0.15 – 0.29	*fewer* disagreeing sources (more sources disagreeing is a worse sign)
ambiguous	0.30 – 0.44	more independent evidence flagging the same field as present-but-unclear
needs_review	0.45 – 0.70	more independent citations agreeing on the same unverified value
extracted	0.85 – 1.00	more independent, verified citations agreeing on the same value
missing	0.00	— no value, nothing to be confident about

The bands never touch, so a `needs_review` field — no matter how much corroborating evidence it has — can never out-rank even a weakly-supported `extracted` one.

```
floor, ceiling = band for this status

if status == "conflicting":
    # more disagreement = worse, so corroboration runs backward
    corroboration = max(SATURATION_CONFUSED - evidence_count, 0) /
(SATURATION_CONFUSED - 2)
else:
    corroboration = max(0, min(evidence_count - 1, SATURATION - 1) / (SATURATION
```

```
  - 1))

  confidence = floor + (ceiling - floor) * corroboration
```

Worked example: `human_oversight` extracted from **3** independently-agreeing, verified citations lands at the top of the `extracted` band — **1.00**. The same field with just one citation would sit at the band's floor, **0.85**.

Class & module reference

MODULE	NAME	WHAT IT'S FOR
`config`	`Settings`	Model names, cache dir, vision on/off — read from `AI_OVERSIGHT_*` env vars via `Settings.from_env()`. Swapping models never touches code.
`parsers.document_ir`	`Element`	One parsed unit (text/key_value/table/chart) with its page, bbox, and content. The common currency every stage passes along.
`parsers.document_ir`	`ParsedDocument`	A document's full `Element` list plus parser warnings. Stage 1's output; Stage 2's input and output; Stage 3's input.
`parsers.pdf_parser / pptx_parser`	`parse()`	File → `ParsedDocument`. No model. Column-aware text, label/value pairing, chart-region and picture-shape detection.
`extractors.granite_vision`	`VisionObservation`	One Granite call's raw response, parsed result, and status (`ok` / `needs_review`).
`extractors.granite_vision`	`run_vision_pass()` / `enrich_document()`	Finds the ambiguous regions, calls Granite, repairs truncated JSON, merges recovered content back into the IR as ordinary elements.
`extractors.granite_vision`	`_extract_scanned_page_with_split_retry()`	A lossy whole-page scan result (truncated or unparseable) gets one retry: split into top/bottom halves, queried separately. Single-level only — a still-lossy half is accepted as-is.
`extractors.granite_vision`	`save_observations()` / `load_observations()`	The caching pair: run the slow vision model once, replay its saved output for as many Stage-3 iterations as needed.
`extractors.rules`	`extract()`	Deterministic Stage-3 backend: field_id/label matching against the canonical 22, with a synonym table.
`extractors.qwen_local`	`extract()`	LLM Stage-3 backend: runs the main chunked pass, then the per-field second pass, then merges both into the result.
`processing.merge`	`collect_metrics()` / `detect_performance_conflicts()`	Backend-agnostic: harvests numeric metrics from chart/table elements and cross-checks a narrative claim against a chart's own numbers.
`processing.validate`	`validate_field_shape()`	Downgrades a value to `needs_review` if its shape doesn't match the field's expectation (e.g. `risk_rating` needs `{inherent, residual}`).

MODULE	NAME	WHAT IT'S FOR
schemas	FieldValue	One field's value, status, evidence list, and computed `confidence`.
schemas	Evidence	One citation: page/slide, element ID, quote, and the method that produced it.
schemas	ExtractionResult	The full output: 22 fields, metrics, incidents, review flags, warnings, and the raw vision observations.
schemas	ExtractionResult.field_match_rate	Computed from `fields`: the fraction of the 22 canonical fields that resolved to any non-null value, not only a confident `extracted` match.
schemas	VisionIntegrity	Computed from `vision_observations`: how many Granite Vision tasks were clean/partially-recovered/failed, a success rate, a pass/review/fail rating, and the specific loss note for each non-clean task.
schemas	ReviewFlag	A rule-generated suggestion for a human — explicitly never a compliance finding.
reporting.json_writer / markdown_writer	write()	Renders a single `ExtractionResult` into the two sibling output files (`extract`'s output).
reporting.csv_writer / html_report_writer	write()	Renders a list of `ExtractionResult`s into one combined report (`batch`'s output) — one row per (file, field), plus each file's `field_match_rate` and, with `--vision`, `vision_integrity`.
cli	parse / vision / extract / batch	The four entry points: structural-only, vision-only (with caching), the full single-file pipeline, and the full pipeline over several files combined into one report.

CLI commands

`ai_oversight parse <file> --out ir.json`

Stage 1 only. No model. Its output JSON is reusable input to any Stage 3 backend later, via that backend's own loader.

`ai_oversight vision <file> --out cache.json`

Stage 2 only, saved to disk. Run once; reuse below as many times as needed.

`ai_oversight extract <file> --backend qwen_local --vision-cache cache.json --out result.json`

The full pipeline, Stage 3 reading from a saved vision cache instead of re-running Granite. Writes `result.json` and `result.review.md`. Drop `--vision-cache` and add `--vision` to run Granite live instead; drop both to skip Stage 2 entirely.

`ai_oversight batch <file1> <file2> ... --backend qwen_local --out report.html`

The same pipeline run independently over several documents, combined into one report: one row per (file, field), with that field's status, value, and confidence. Written as CSV, or as a styled HTML report when `--out` ends in `.html` — no extra flag needed. A file that fails to parse or extract is reported and skipped rather than aborting the whole batch. Every row also carries that document's `field_match_rate` — the fraction of the 22 canonical fields that resolved to *any* non-null value, not only a confident `extracted` match (a `needs_review` / `ambiguous` / `conflicting` field with a real value still counts as matched; only a field with no value at all counts as unmatched). With `--vision`, every row also carries that document's `vision_integrity` rating, success rate, and any Granite Vision data-loss notes (CSV: `field_match_rate` / `vision_rating` / `vision_success_rate` / `vision_data_loss` columns; HTML: a field-match-rate line plus a pass/review/fail vision callout under each file's heading).

Testing with Ground-truth Check

Every file in this section is a **simulated** PDF or PPTX report — synthetic, fictional test documents built to exercise this pipeline, not real filings from any organization. In total, **6 simulated documents** are tested: two minimal single-page/slide vision-only fixtures, and four larger, more realistic reports. Each one is run through the real pipeline via `batch` (live Granite Vision + `qwen_local`), and every resulting row is then checked by hand against that document's actual content: does this field's information really appear in the source, or not?

Two deliberately minimal files start this off — `Mini_Vision_Test_Internal_AI_Risk.pdf` (1 scanned page, 10 label/value facts) and `Mini_Vision_Test_External_AI_Usage.pptx` (1 slide, 9 facts) — producing 44 rows (22 fields × 2 files).

37 of 44 rows are clean matches — the pipeline's `missing` correctly means "not stated anywhere," and its `extracted` correctly traces to a real, unambiguous statement. **6** are *Partial* cases: the source states something related but incomplete (e.g. only a residual risk rating, no inherent one) or one statement reasonably covers two related canonical fields (a single generic

"Owner" line for both `business_owner` and `technical_owner`). **1** is the interesting case — `next_review_date` on the PDF — where the model inferred a plausible-looking date from the validation statement's own date, a genuinely different fact; the substantiation check caught the mismatch and correctly held it at `needs_review` instead of letting it through as `extracted`.

File-level summary

Gross Field Match (22 fields) is this pipeline's own `field_match_rate`: the fraction of all 22 canonical fields that resolved to *any* non-null value — a `needs_review`, `ambiguous`, or `conflicting` field still counts as matched here as long as it carries a real value for a reviewer to look at; only a field with no value at all (`missing`, or a status that settled on nothing, e.g. an `ambiguous` field too unresolved to produce even a tentative value) counts as unmatched. **Ground-truth Field Match (excl. not-mentioned)** narrows the denominator to only the fields the ground-truth check confirms are genuinely present in that source document (`Yes` or `Partial`) — a field ground truth says is truly absent (`Not stated`) and the pipeline correctly reports `missing` is a correct outcome, but counting it in a "how much did we successfully match" denominator would understate the rate for documents that simply state fewer of the 22 facts. **Granite parsing for image/chart** is this run's `vision_integrity` rating (see Stage 2 above); `N/A` means the file has a native text layer and Granite Vision never ran.

FILE	GRANITE PARSING FOR IMAGE/CHART	GROSS FIELD MATCH (22 FIELDS)	GROUND-TRUTH FIELD MATCH (EXCL. NOT-MENTIONED)
Mini_Vision_Test_Internal_AI_Risk.pdf	PASS	0.59 (13/22)	1.00 (12/12)
Mini_Vision_Test_External_AI_Usage.pptx	PASS	0.50 (11/22)	1.00 (11/11)
Sample_Internal_AI_Oversight_Report.pdf	N/A	1.00 (22/22)	1.00 (22/22)
Sample_External_AI_Usage_Disclosure.pptx	N/A	1.00 (22/22)	1.00 (22/22)
Vision_Required_Internal_AI_Risk_Report.pdf	PASS	0.73 (16/22)	0.94 (15/16)
Vision_Required_External_AI_Usage_Disclosure.pptx	PASS	0.77 (17/22)	0.94 (16/17)

Mini_Vision_Test_Internal_AI_Risk.pdf (1 page)

`Granite Vision` this file has no native text layer (fully scanned) -- every fact below, including every **Source** quote, was recovered via Granite Vision's `tables_json` task, not parsed natively.

FIELD	STATUS	VALUE	CONF.	GROUND TRUTH	SOURCE
submission_id	missing		0.0	Not stated	
reporting_entity	missing		0.0	Not stated	
application_name	missing		0.0	Not stated	
business_use_case	extracted	Settlement exception prediction	0.85	Yes	Granite Vision Use case: Settlement exception prediction
business_owner	extracted	Post-Trade Operations	0.85	Partial	Granite Vision Owner: Post-Trade Operations (one generic “Owner” row, not split business/technical)
technical_owner	extracted	Post-Trade Operations	0.85	Partial	Granite Vision Owner: Post-Trade Operations (same row reused)
model_type	extracted	Gradient-boosted trees	0.85	Yes	Granite Vision Model type: Gradient-boosted trees
model_provider	extracted	Internal / in-house	0.85	Yes	Granite Vision Provider: Internal / in-house
model_version	missing		0.0	Not stated	
lifecycle_status	missing		0.0	Not stated	
first_deployment_date	missing		0.0	Not stated	
input_data	extracted	Confidential transaction records	0.85	Partial	Granite Vision Data sensitivity: Confidential transaction records (no distinct data-sources statement)
data_sensitivity	extracted	Confidential transaction records	0.85	Yes	Granite Vision Data sensitivity: Confidential transaction records
financial_criticality	extracted	HIGH	0.85	Yes	Granite Vision Financial criticality: HIGH
human_oversight	extracted	Analyst approves any action	0.85	Yes	Granite Vision Human oversight: Analyst approves any action
validation_status	extracted	Passed independent testing, Oct 2026	0.85	Yes	Granite Vision Validation results: Passed independent testing, Oct 2026
model_performance	missing		0.0	Not stated	
risk_rating	needs_review	MEDIUM	0.45	Partial	Granite Vision Residual risk rating: MEDIUM (no inherent rating stated)
incident_history	extracted	1 low-severity alert issue, Sep 2026	0.85	Yes	Granite Vision Incident history: 1 low-severity alert issue, Sep 2026
monitoring_frequency	missing		0.0	Not stated	
fallback_process	missing		0.0	Not stated	

FIELD	STATUS	VALUE	CONF.	GROUND TRUTH	SOURCE
next_review_date	needs_review	2026-10-01	0.45	Not stated	Granite Vision model inferred this from the Oct 2026 validation date -- a different fact; correctly caught as needs_review, not extracted

Mini_Vision_Test_External_AI_Usage.pptx (1 slide)

Granite Vision this slide is a single picture shape -- every fact below, including every **Source** quote, was recovered via Granite Vision's `tables_json` task, not parsed natively.

FIELD	STATUS	VALUE	CONF.	GROUND TRUTH	SOURCE
submission_id	missing		0.0	Not stated	
reporting_entity	extracted	HarborView Operations (fictional)	0.85	Yes	Granite Vision Reporting organization: HarborView Operations (fictional)
application_name	missing		0.0	Not stated	
business_use_case	missing		0.0	Not stated	
business_owner	missing		0.0	Not stated	
technical_owner	missing		0.0	Not stated	
model_type	extracted	Hosted instruction LLM	0.85	Yes	Granite Vision Model / provider: Hosted instruction LLM / Northstar AI
model_provider	extracted	Northstar AI	0.85	Yes	Granite Vision Model / provider: Hosted instruction LLM / Northstar AI
model_version	missing		0.0	Not stated	
lifecycle_status	missing		0.0	Not stated	
first_deployment_date	missing		0.0	Not stated	
input_data	extracted	Restricted, masked customer fields	0.85	Partial	Granite Vision Data classification: Restricted, masked customer fields (no distinct data-sources statement)
data_sensitivity	extracted	Restricted, masked customer fields	0.85	Yes	Granite Vision Data classification: Restricted, masked customer fields
financial_criticality	extracted	MEDIUM	0.85	Yes	Granite Vision Financial criticality: MEDIUM
human_oversight	extracted	Mandatory approval before any action	0.85	Yes	Granite Vision Human oversight: Mandatory approval before any action
validation_status	extracted	Passed review, October 2026	0.85	Yes	Granite Vision Validation results: Passed review, October 2026

FIELD	STATUS	VALUE	CONF.	GROUND TRUTH	SOURCE
model_performance	missing		0.0	Not stated	
risk_rating	needs_review	MEDIUM	0.45	Partial	Granite Vision Residual risk rating: MEDIUM (no inherent rating stated)
incident_history	extracted	0 reportable in past 90 days	0.85	Yes	Granite Vision Incidents: 0 reportable in past 90 days
monitoring_frequency	extracted	Monthly	0.85	Yes	Granite Vision Monitoring frequency: Monthly
fallback_process	missing		0.0	Not stated	
next_review_date	missing		0.0	Not stated	

Sample_Internal_AI_Oversight_Report.pdf (4 pages)

Every one of the 22 fields genuinely exists here, cleanly labeled in an explicit field-ID inventory on page 2 ("22 mapped fields • explicit IDs support extraction testing") -- so **Ground truth** is Yes throughout, and after fixing two real precision bugs found via this exact fixture (preferring a structured element's own label over a bare fallback value, and preferring an explicit key_value citation over a disagreeing narrative mention elsewhere in the document), the pipeline now resolves **all 22 of 22** fields correctly.

FIELD	STATUS	VALUE	CONF.	GROUND TRUTH	SOURCE
submission_id	extracted	AI-INT-2026-041	0.93	Yes	Submission ID: AI-INT-2026-041
reporting_entity	extracted	Example Post-Trade Operations (fictional)	0.85	Yes	Legal entity: Example Post-Trade Operations (fictional)
application_name	extracted	Settlement Failure Early Warning (SFEW)	0.93	Yes	Application name: Settlement Failure Early Warning (SFEW)
business_use_case	extracted	T+1 settlement exception prediction	0.93	Yes	Business use case: T+1 settlement exception prediction
business_owner	extracted	Maya Chen, Director, Post-Trade Analytics	0.85	Yes	Business owner: Maya Chen, Director, Post-Trade Analytics
technical_owner	extracted	Alex Rivera, Senior ML Engineer	0.85	Yes	Technical owner: Alex Rivera, Senior ML Engineer
model_type	extracted	Gradient-boosted trees (XGBoost)	0.85	Yes	Model type: Gradient-boosted trees (XGBoost)
model_provider	extracted	Internally developed; no external inference API	0.85	Yes	AI provider: Internally developed; no external inference API

FIELD	STATUS	VALUE	CONF.	GROUND TRUTH	SOURCE
model_version	extracted	2.3.1	0.93	Yes	Model version: 2.3.1
lifecycle_status	extracted	Production	0.93	Yes	Lifecycle status: Production
first_deployment_date	extracted	2026-07-15	0.85	Yes	First deployment: 2026-07-15
input_data	extracted	Transaction metadata + exception history	0.85	Yes	Input data: Transaction metadata + exception history
data_sensitivity	extracted	Confidential, transaction-level data	0.93	Yes	Data sensitivity: Confidential, transaction-level data
financial_criticality	extracted	High; advisory operations prioritization	0.85	Yes	Financial criticality: High; advisory operations prioritization
human_oversight	extracted	Analyst approves every escalation	1.0	Yes	Human oversight: Analyst approves every escalation
validation_status	extracted	Passed independent validation on 2026-10-20	0.93	Yes	Validation status: Passed independent validation on 2026-10-20
model_performance	extracted	{'precision': 0.91, 'recall': 0.86, 'roc_auc': 0.95}	1.0	Yes	Performance: Precision 0.91; recall 0.86; ROC-AUC 0.95
risk_rating	extracted	{'inherent': 'HIGH', 'residual': 'MEDIUM'}	1.0	Yes	Risk rating: Inherent HIGH; residual MEDIUM
incident_history	extracted	1 closed incident; see page 4	1.0	Yes	Incident history: 1 closed incident; see page 4
monitoring_frequency	extracted	Daily metrics; monthly drift review	1.0	Yes	Monitoring frequency: Daily metrics; monthly drift review
fallback_process	extracted	Manual rules-based exception queue	0.93	Yes	Fallback process: Manual rules-based exception queue
next_review_date	extracted	2027-01-20	0.85	Yes	Next periodic review: 2027-01-20

Sample_External_AI_Usage_Disclosure.pptx (7 slides)

Same pattern as the PDF above -- an explicit field-ID inventory across slides 3-4 means every field is genuinely present (Yes throughout), and after the same two fixes, the pipeline resolves **all 22 of 22** fields correctly here too.

FIELD	STATUS	VALUE	CONF.	GROUND TRUTH	SOURCE
submission_id	extracted	EXT-2026-118	0.93	Yes	Submission ID: EXT-2026-118
reporting_entity	extracted	Meridian Clearing Services (fictional)	0.93	Yes	Reporting organization: Meridian Clearing Services (fictional)
application_name	extracted	Collateral Exception Triage Copilot (CETC)	0.85	Yes	AI application: Collateral Exception Triage Copilot (CETC)
business_use_case	extracted	Collateral exception classification + routing	1.0	Yes	Business use case: Collateral exception classification + routing
business_owner	extracted	Priya Nair, Director of Operations Risk	0.85	Yes	Business owner: Priya Nair, Director of Operations Risk
technical_owner	extracted	Ethan Brooks, ML Platform Lead	0.85	Yes	Technical owner: Ethan Brooks, ML Platform Lead
model_type	extracted	Retrieval-augmented LLM + classifier	0.93	Yes	Model type: Retrieval-augmented LLM + classifier
model_provider	extracted	NimbusAI Cloud (fictional hosted provider)	0.93	Yes	AI / model provider: NimbusAI Cloud (fictional hosted provider)
model_version	extracted	CETC v1.7; NimbusText-4.2	0.85	Yes	Version: CETC v1.7; NimbusText-4.2
lifecycle_status	extracted	Controlled rollout / analyst-in-the-loop	0.85	Yes	Deployment status: Controlled rollout / analyst-in-the-loop
first_deployment_date	extracted	2026-08-04	0.85	Yes	First deployed: 2026-08-04
input_data	extracted	Tokenized tickets, settlement messages, runbooks	0.93	Yes	Data inputs: Tokenized tickets, settlement messages, runbooks
data_sensitivity	extracted	Confidential; direct identifiers tokenized	1.0	Yes	Data sensitivity: Confidential; direct identifiers tokenized
financial_criticality	extracted	Moderate; human-controlled collateral operations	0.85	Yes	Financial criticality: Moderate; human-controlled collateral operations
human_oversight	extracted	Analyst must approve every classification & action	1.0	Yes	Human oversight: Analyst must approve every classification & action
validation_status	extracted	2026-10-08; 600-case independent review	1.0	Yes	Validation: 2026-10-08; 600-case independent review
model_performance	extracted	{'accuracy': 0.92, 'grounded_response_rate': 0.97}	1.0	Yes	Model performance: Accuracy 92%; grounded response rate 97%

FIELD	STATUS	VALUE	CONF.	GROUND TRUTH	SOURCE
risk_rating	extracted	{'inherent': 'HIGH', 'residual': 'MEDIUM'}	1.0	Yes	Risk rating: Inherent HIGH; residual MEDIUM
incident_history	extracted	2 resolved incidents (Sep 07, Oct 02, 2026)	1.0	Yes	Incident history: 2 resolved incidents (Sep 07, Oct 02, 2026)
monitoring_frequency	extracted	Daily prompt logs; weekly QA sampling	1.0	Yes	Monitoring frequency: Daily prompt logs; weekly QA sampling
fallback_process	extracted	Manual triage queue + static decision tree	1.0	Yes	Fallback process: Manual triage queue + static decision tree
next_review_date	extracted	2027-01-08	0.93	Yes	Next review date: 2027-01-08

Vision_Required_Internal_AI_Risk_Report.pdf (4 pages)

The original, full-size fixture that motivated Phase 3: every page is a scanned image with no native text layer, so every row below comes from Granite Vision. Most facts are genuinely present and cleanly stated (13 of 22 fields) -- these mostly resolve correctly. Three fields (`risk_rating` , `monitoring_frequency` , `fallback_process`) are only *Partial*: the document states something related but incomplete (e.g. a residual rating with no inherent one) or buried inside one row of a 5-row control matrix rather than stated as a document-level fact. The remaining 6 are genuinely absent. Two results are worth calling out specifically: `technical_owner` has no real answer in this document at all, and the structural table-citation check correctly stopped an irrelevant control-matrix citation from being confidently accepted; `monitoring_frequency` came back `missing` this run even though "Monthly PSI review" is a real (if buried) signal -- the per-field second pass has surfaced it in some runs and not others, a known soft spot for single-row facts hidden inside a multi-record table.

FIELD	STATUS	VALUE	CONF.	GROUND TRUTH	SOURCE
submission_id	missing		0.0	Not stated	
reporting_entity	missing		0.0	Not stated	
application_name	extracted	Settlement Failure Early Warning	0.85	Yes	Granite Vision APPLICATION: Settlement Failure Early Warning
business_use_case	extracted	Predict and prioritize likely US equity settlement failures	0.85	Yes	Granite Vision USE CASE: Predict and prioritize likely US equity settlement failures
business_owner	needs_review	Maya Chen, Director of Settlement Risk	0.45	Yes	Granite Vision BUSINESS OWNER: Maya Chen, Director of Settlement Risk
technical_owner	needs_review	Operations Lead	0.45	Not stated	Granite Vision no distinct technical-owner row exists; the model's "Operations Lead" citation is a control's owner in an unrelated 5-row control matrix

FIELD	STATUS	VALUE	CONF.	GROUND TRUTH	SOURCE
model_type	extracted	LightGBM gradient boosting + rule guardrails	0.85	Yes	Granite Vision AI/MODEL TYPE: LightGBM gradient boosting + rule guardrails
model_provider	extracted	Internally developed / LightGBM v4.5.0	0.85	Yes	Granite Vision MODEL PROVIDER: Internally developed / LightGBM v4.5.0
model_version	extracted	r2.3.1	0.85	Yes	Granite Vision MODEL VERSION: r2.3.1
lifecycle_status	extracted	Production	0.85	Yes	Granite Vision DEPLOYMENT STATUS: Production
first_deployment_date	needs_review	2026-04-15	0.45	Yes	Granite Vision PRODUCTION DATE: 15 April 2026
input_data	extracted	Synthetic settlement events; exception queues; counterparty flags	0.85	Yes	Granite Vision DATA SOURCES: Synthetic settlement events; exception queues; counterparty flags
data_sensitivity	extracted	Confidential market participant transaction data	0.85	Yes	Granite Vision DATA CLASSIFICATION: Confidential market participant transaction data
financial_criticality	extracted	HIGH - impacts exception prioritization	0.85	Yes	Granite Vision FINANCIAL CRITICALITY: HIGH - impacts exception prioritization
human_oversight	extracted	Operations analyst signs off on interventions	0.85	Yes	Granite Vision HUMAN OVERSIGHT: Operations analyst signs off on interventions
validation_status	missing		0.0	Not stated	
model_performance	extracted	{'precision': 0.9109999999999999, 'recall': 0.87, 'samples': 1000.0}	0.85	Yes	Granite Vision n=1,000 records | Precision 91.1% | Recall 87.0%
risk_rating	needs_review	MEDIUM	0.45	Partial	Granite Vision RESIDUAL RISK: MEDIUM (no inherent rating stated)
incident_history	missing		0.0	Not stated	
monitoring_frequency	missing		0.0	Partial	Granite Vision C-03 | Monthly PSI review (one control among 5, not a document-level statement)
fallback_process	needs_review	Manual fallback drill	0.45	Partial	Granite Vision C-04 | Manual fallback drill (one control among 5, not a document-level statement)
next_review_date	missing		0.0	Not stated	

Vision_Required_External_AI_Usage_Disclosure.pptx (6 slides)

Same vision-only character as the PDF above. All **12** fields the pipeline marks `extracted` are genuine, correct matches, and **4 of 5** genuinely absent fields are correctly `missing` -- the fifth, `technical_owner`, is again a

safety-net catch (an irrelevant control-matrix citation correctly held at `needs_review` instead of accepted). After the key_value-citation fix, `risk_rating` no longer comes back `conflicting` : the model now synthesizes both the inherent and residual ratings into one combined answer, correctly held at `needs_review` rather than `extracted` since its own citation only points to one of the two source rows. One open limitation remains: `incident_history` comes back `ambiguous` rather than a confident count, since the model can see a real 3-row incident table but doesn't confidently aggregate it into one document-level answer.

FIELD	STATUS	VALUE	CONF.	GROUND TRUTH	SOURCE
submission_id	extracted	EXT-MRG-2026-031	0.85	Yes	Granite Vision APPLICATION ID: EXT-MRG-2026-031
reporting_entity	extracted	Riverton Clearing Services (fictional)	0.85	Yes	Granite Vision SUBMITTING FIRM: Riverton Clearing Services (fictional)
application_name	extracted	Margin Call Support Copilot	0.85	Yes	Granite Vision APPLICATION NAME: Margin Call Support Copilot
business_use_case	extracted	Draft margin-call explanations and suggest resolution codes	0.93	Yes	Granite Vision BUSINESS USE CASE: Draft margin-call explanations and suggest resolution codes
business_owner	extracted	Elena Ortiz, VP Collateral Services	0.85	Yes	Granite Vision BUSINESS OWNER: Elena Ortiz, VP Collateral Services
technical_owner	needs_review	AI Governance	0.45	Not stated	Granite Vision no distinct technical-owner row exists; the model's "AI Governance" citation is a control's owner in an unrelated 5-row control matrix
model_type	extracted	Retrieval-augmented LLM, instruction tuned	0.85	Yes	Granite Vision MODEL TYPE: Retrieval-augmented LLM, instruction tuned
model_provider	extracted	Self-hosted Qwen2.5-7B-Instruct	0.85	Yes	Granite Vision MODEL PROVIDER: Self-hosted Qwen2.5-7B-Instruct
model_version	extracted	mc-support-v1.6.3	0.85	Yes	Granite Vision MODEL VERSION: mc-support-v1.6.3
lifecycle_status	needs_review	PILOT	0.45	Partial	Granite Vision "PILOT" appears in a garbled, misaligned vision-recovered table, not a clean document-level statement
first_deployment_date	missing		0.0	Not stated	
input_data	extracted	Approved margin statements, collateral rules, dispute history	0.85	Yes	Granite Vision DATA SOURCES: Approved margin statements, collateral rules, dispute history
data_sensitivity	extracted	RESTRICTED - client margin and account-level data	0.85	Yes	Granite Vision DATA SENSITIVITY: RESTRICTED - client margin and account-level data
financial_criticality	extracted	HIGH - supports same-day margin exception workflows	0.93	Yes	Granite Vision FINANCIAL CRITICALITY: HIGH - supports same-day margin exception workflows

FIELD	STATUS	VALUE	CONF.	GROUND TRUTH	SOURCE
human_oversight	extracted	Mandatory analyst approval for all client messages	1.0	Yes	Granite Vision HUMAN OVERSIGHT: Mandatory analyst approval for all client messages
validation_status	missing		0.0	Not stated	
model_performance	needs_review	{'triage_accuracy': '86%', 'grounded_response_rate': '92%', 'accuracy_target': '90%'}	0.45	Yes	Granite Vision Triage accuracy/Grounded response rate chart: 86%/92%, 89%/95% vs. 90% target
risk_rating	needs_review	INHERENT RISK: HIGH, RESIDUAL RISK: MEDIUM	0.7	Yes	Granite Vision INHERENT RISK: HIGH / RESIDUAL RISK: MEDIUM (two separate rows, both stated)
incident_history	ambiguous		0.3	Yes	Granite Vision 3-row incident table: INC-103, INC-102, INC-101
monitoring_frequency	needs_review	Daily quality check / monthly review	0.45	Partial	Granite Vision Model drift | Daily quality check / monthly review (one control among 5, not a document-level statement)
fallback_process	missing		0.0	Not stated	
next_review_date	missing		0.0	Not stated	

---

ai_oversight — local AI-governance-report extraction. Every `extracted` value traces to real source text; everything else is marked for a human to look at.